COLLECTIONS MODULE

DEQUE:

→it is preferred over list in the cases where we need quicker append and pop operations from both the ends of container.

→it provides an O(1) time complexity for append and pop operations are compared to list which provides O(n) time complexity.

Functions:

1.append(): it is used to insert the values in its arguments to the right end of deque.

2.appendleft(): it is used to insert the value in its arguments to the left end of deque.

3.pop(): it is used to delete an argument from the right end of deque.

4.popleft(): it is used to delete an argument from the left end of deque.

Example:

```
>>> import collections
>>> de=collections.deque([4,5,6])
>>> de.append(7)
>>> print(de)
deque([4, 5, 6, 7])
>>> de.appendleft(23)
>>> print(de)
deque([23, 4, 5, 6, 7])
>>> de.pop()
7
>>> print(de)
deque([23, 4, 5, 6])
>>> de.popleft()
23
>>> print(de)
deque([4, 5, 6])
>>>
```

5.index(ele,beg,end): this function returns the index of the value mentioned in arguments , storing searching from beg till end index.

6.insert(I,a): this function inserts the value mentioned in arguments at index specified in arguments.

7.remove(): it removes the first occurrence of value mentioned in arguments

8.count(): it counts no.of occurrences of value mentioned in arguments.

Example:

```
>>> import collections
>>> de=collections.deque([1,2,4,3,6,4,2])
>>> print(de.index(6,1,6))
4
>>> de.insert(4,3)
```

```
>>> print(de)
deque([1, 2, 4, 3, 3, 6, 4, 2])
>>> print(de.count(3))
2
>>> print(de.count(1))
1
>>>
>>> de.remove(6)
>>> print(de)
deque([1, 2, 4, 3, 3, 4, 2])
>>>
```

9.extend(iterable): it is used to add multiple values at the right end of deque. The argument passed is an iterable.
10.extendleft(iterable): it is used to add multiple values at the left end of deque. the argument passed is an iterable. order is reversed as a result to left appends.
11.reverse():
it is used to reverse order of elements.
12.rotate():
it rotates the deque by the number specified in arguments. if the number specified is negative , rotation occurs to left. else rotation is to right.
Example:

```
>>> import collections
>>> de=collections.deque([10,11,12])
>>> de.extend([15,16,17])
>>> print(de)
deque([10, 11, 12, 15, 16, 17])
>>> de.extendleft([20,21,22])
>>> print(de)
deque([22, 21, 20, 10, 11, 12, 15, 16, 17])
>>> de.rotate(-3)
>>> print(de)
deque([10, 11, 12, 15, 16, 17, 22, 21, 20])
>>> de.reverse()
>>> print(de)
deque([20, 21, 22, 17, 16, 15, 12, 11, 10])
>>>
```

NAMEDTUPLE:

→it is present in 'collections' module.
→it is similar to dictionaries.
→it contains keys that are hashed to a particular value.

→it supports both access from key value and iteration, the functionality that dictionaries lack.
Functions:
1.Acessing as index:
the attribute values of namedtuple() are ordered and can be accessed using the index number unlike dictionaries which are not accessible by index.
2.Access by keyname:
it is also allowed as in dictionaries
3.getattr():
it access the value by giving namedtuple and key value as its arguments.
Example:

```
>>> import collections
>>> student=collections.namedtuple('student',['name','age','DOB'])
>>> s=student('john','32','09051990')
>>> print(s[1])
32
>>> print(s.DOB)
09051990
>>> print(getattr(s,'name'))
john
>>>
```

conversion functions:
1._make(): it is used to return a namedtuple() from the iterable passed as argument.
2._asdict():it returns the orderDict() as constructed from the mapped values of namedtuple.
Example:

```
>>> import collections
>>> student=collections.namedtuple('student',['name','age','DOB'])
>>> s=student('john','32','09051990')
>>> print(s._asdict())
OrderedDict([('name', 'john'), ('age', '32'), ('DOB', '09051990')])
>>> li=['anil','31','17051991']
>>> print(student._make(li))
student(name='anil', age='31', DOB='17051991')
>>>
```

additional functions:
1._fields():
it is used to return all the keynames of the namespace declared
2._replace():
it is used to change the values mapped with the passed keyname.
Example:

```
>>> import collections
>>> customers=collections.namedtuple('customers',['cname','product','city'])
```

```
>>> c=customers('john','hp','hyd')
>>> print(c._fields)
('cname', 'product', 'city')
>>> print(c._replace(cname='richard'))
customers(cname='richard', product='hp', city='hyd')
>>>
```

OS module

it provides a portable way of using operating system dependent functionality.

Functions:

1.os.name:

it gives the name of the operating system dependent module.

the following names have currently been registered:'posix','nt','os2','ce','java', and 'riscos'.

2.os.getcwd():

it returns the current working directory of the file used to execute the code, can vary from system to system.

Example:

```
>>> import os
>>> print(os.name)
nt
>>> print(os.getcwd())
C:\Users\Y.CHARAN RAJ\AppData\Local\Programs\Python\Python37
>>>
```

File object manipulation:

1.os.popen():

it opens a pipe to or from command. The return value can be read or written depending on whether mode is 'r' or 'w'.

Syntax:

```
os.popen(command[,mode[,bufsize]])
```

2.os.close():

this function is used to close the file.

Example:

```
import os
fd="input.txt"
file=open(fd,'w')
file.write("hello world")
file.close()
file=open(fd,'r')
text=file.read()
```

```
print(text)
file=os.popen(fd,'w')
file.write("hello world")
```

3.os.error:

All functions in this module raise OSError in the case of invalid or inaccessible file names and paths or other arguments that have the correct type, but are not accepted by the operating system.

Example:

```
 import os
try:
   filename="output.txt"
   f=open(filename,'r')
   text=f.read()
   f.close()
except IOError:
   print("problem reading:"+filename)
```

4.os.rename():

this function can remove a filename, a file old.txt can be renamed to new.txt.

Example:

```
>> import os
>>> fd="input.txt"
>>> os.rename(fd,"output.txt")
>>>
```

RANDOM NUMBERS

To implement random numbers functions we have to use 'random pre-defined module'

This concept is used in games, lotteries or any application requiring random number generation.

Functions:

1.choice()

it is used to generate one random number from a container.

2.randrange(beg,end,step):

it is used to find random number with in the given range.

Note:

beg→include in generation

end→excluded in generation

step→to skip numbers in range while selecting

Example:

```
>>> import random
>>> random.choice([23,34,78,12,90,123,87,43])
90
>>> random.choice([23,34,78,12,90,123,87,43])
```

```
78
>>> random.choice([23,34,78,12,90,123,87,43])
43
>>> print(random.randrange(30,40,4))
30
>>> print(random.randrange(30,40,4))
34
>>> print(random.randrange(30,40,4))
38
>>>
```

3.random():
it is used to generate a float random number less than one and greater or equal to 'zero'.
4.seed():
it maps a particular random number with the seed argument number. All the random number called after the seeded value returns the mapped number.
Example:

```
>>> import random
>>> print(random.random())
0.5578274736396559
>>> random.seed(9)
>>> print(random.random())
0.46300735781502145
>>> random.seed(5)
>>> print(random.random())
0.6229016948897019
>>>
```

5.shuffle():
it is used to shuffle the entire list to randomly arrange them.
6.uniform(a,b):
it is used to generate a floating point random number between the numbers mentioned in its arguments.
Example:

```
import random
list=[1,4,5,110,2]
for i in range(0,len(list)):
   print(list[i],end="\t")
print()
random.shuffle(list)
print("the list after shuffling is:")
for i in range(0,len(list)):
```

```
    print(list[i],end="\t")
print()
print("the random floating point numbers between 5 and 10 is:")
print(random.uniform(5,10))
```